

TP 2: Pretraitement des données

October 8, 2023

1 Introduction

L'exploration et le prétraitement des données sont des étapes cruciales dans le processus d'analyse de données et de création de modèles prédictifs. Pour que nos modèles puissent fournir des résultats précis et significatifs, il est impératif de comprendre, nettoyer et préparer nos données en conséquence. Dans ce TP, nous allons plonger profondément dans l'art du prétraitement des données, en mettant l'accent sur plusieurs concepts essentiels.

Objectifs du TP :

1. **Compréhension des données** : Notre première étape consistera à comprendre les données avec lesquelles nous travaillons. Nous examinerons les différentes caractéristiques, identifierons les valeurs manquantes ou aberrantes, et explorerons les corrélations potentielles entre les variables.
2. **Imputation des Données** : Face à des valeurs manquantes, il est essentiel de décider comment les traiter. Nous allons découvrir l'utilisation d'imputeurs pour remplir ces lacunes de manière intelligente, en préservant l'intégrité de nos données.
3. **Encodage des Variables Catégorielles** : Les algorithmes d'apprentissage automatique requièrent généralement des données numériques. Nous aborderons les techniques d'encodage pour convertir des variables catégorielles en formats appropriés, tout en évitant les biais indésirables.
4. **Normalisation et Mise à l'Échelle** : Les données aux échelles différentes peuvent entraîner des problèmes de convergence et d'interprétation. Nous étudierons les méthodes de mise à l'échelle pour garantir que toutes les caractéristiques aient un impact équitable dans la modélisation.
5. **Make Column Transform et Pipelines** : Nous apprendrons comment simplifier et automatiser le processus de prétraitement en utilisant `make_column_transformer` et en construisant des pipelines. Cela rendra notre flux de travail plus efficace et plus maintenable.

Au terme de ce TP, vous serez en mesure de comprendre et d'appliquer des techniques essentielles de prétraitement des données, ce qui est indispensable pour mener à bien des projets d'apprentissage automatique. Préparez-vous à explorer, nettoyer et préparer vos données de manière experte pour des analyses et des modélisations réussies.

2 Imputers

Dans le contexte de l'analyse des données, la gestion des valeurs manquantes est une étape cruciale pour garantir la fiabilité des résultats. Les valeurs manquantes, souvent issues de diverses sources telles que les erreurs de mesure ou les données non disponibles, peuvent fausser les analyses et les modèles prédictifs. Pour aborder ce défi, différentes stratégies sont mises en œuvre. Dans le code ci-dessus, quatre de ces stratégies sont illustrées.

- La première stratégie, basée sur la moyenne, remplace les valeurs manquantes par la moyenne des valeurs disponibles dans la même colonne.
- La deuxième, reposant sur la médiane, utilise la médiane comme valeur de remplacement.
- La troisième stratégie, dite “la plus fréquente”, remplace les valeurs manquantes par le mode, c'est-à-dire la valeur la plus courante dans la colonne.
- Enfin, la quatrième stratégie adopte une approche plus spécifique en remplaçant les valeurs manquantes par une valeur constante, ici 99.

Chacune de ces stratégies présente des avantages et des inconvénients. L'utilisation de la moyenne ou de la médiane est adaptée aux distributions continues, tandis que la stratégie “la plus fréquente” convient aux variables catégorielles. La stratégie de valeur constante peut être utile pour conserver l'information sur les valeurs manquantes. Cependant, elle peut introduire des biais si elle est utilisée à l'excès. L'adoption de la stratégie appropriée dépend de la nature des données et des objectifs de l'analyse.

Ce code illustre les différentes stratégies à l'aide de la bibliothèque Scikit-Learn. Les fonctions `fit_transform` remplacent les valeurs manquantes selon les stratégies spécifiées. Il est important de choisir la stratégie qui convient le mieux à la distribution des données et aux objectifs analytiques.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer

X = np.array([[10, 3],
              [3, 5],
              [5, 6],
              [np.nan, 3]])

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer2 = SimpleImputer(missing_values=np.nan, strategy='median')
imputer3 = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
imputer4 = SimpleImputer(missing_values=np.nan, strategy='constant',
                           fill_value=99)

print("Mean Strategy:")
print(imputer.fit_transform(X))

print("Median Strategy:")
print(imputer2.fit_transform(X))

print("Most Frequent Strategy:")
```

```
print(imputer3.fit_transform(X))

print("Constant Strategy:")
print(imputer4.fit_transform(X))
```

Mean Strategy:

```
[[10.  3.]
 [ 3.  5.]
 [ 5.  6.]
 [ 6.  3.]]
```

Median Strategy:

```
[[10.  3.]
 [ 3.  5.]
 [ 5.  6.]
 [ 5.  3.]]
```

Most Frequent Strategy:

```
[[10.  3.]
 [ 3.  5.]
 [ 5.  6.]
 [ 3.  3.]]
```

Constant Strategy:

```
[[10.  3.]
 [ 3.  5.]
 [ 5.  6.]
 [99.  3.]]
```

3 Pre-processing

L'introduction aux étapes de prétraitement des données occupe une place cruciale dans la phase préliminaire de toute analyse de données. Le prétraitement consiste en une série de procédures visant à nettoyer, transformer et préparer les données brutes en vue de leur utilisation ultérieure dans des modèles d'analyse. Deux techniques fondamentales de prétraitement seront abordées dans cet exposé : l'encodage et la standardisation.

3.1 Encodage

L'encodage sert à convertir des variables catégorielles en format numérique, rendant ainsi les données compatibles avec les algorithmes d'apprentissage automatique.

3.1.1 LabelEncoder :

Le LabelEncoder est une technique d'encodage qui transforme les valeurs de variables catégorielles en valeurs numériques discrètes. Il attribue des étiquettes numériques uniques à chaque catégorie de la variable, permettant ainsi de représenter les catégories sous forme numérique. Cependant, il convient de noter que cette méthode peut introduire un ordre implicite entre les catégories, ce qui peut être inapproprié pour certaines variables catégorielles sans relation d'ordre intrinsèque. Le LabelEncoder est généralement utilisé pour encoder des variables cibles ou des variables catégorielles ordinales.

```
[2]: from sklearn.preprocessing import LabelEncoder,
      ↪LabelBinarizer, MultiLabelBinarizer, OrdinalEncoder, OneHotEncoder
```

```
[3]: y = np.array(['chat', 'chien', 'chat', 'oiseau', 'lion', 'chien', 'chat'])

encoder = LabelEncoder()
print(encoder.fit_transform(y))
print(encoder.inverse_transform(np.array([0, 0, 2, 3, 3, 3, 0, 1, 2])))
```

```
[0 1 0 3 2 1 0]
```

```
['chat' 'chat' 'lion' 'oiseau' 'oiseau' 'oiseau' 'chat' 'chien' 'lion']
```

3.1.2 LabelBinarizer et MultiLabelBinarizer :

Le LabelBinarizer et le MultiLabelBinarizer sont utilisés pour transformer des variables catégorielles en une matrice binaire. Le LabelBinarizer est utilisé pour les variables catégorielles binaires, tandis que le MultiLabelBinarizer traite les variables catégorielles pouvant prendre plusieurs valeurs pour une seule observation. Ces techniques créent des colonnes binaires distinctes pour chaque catégorie et indiquent la présence ou l'absence de chaque catégorie en utilisant des valeurs binaires (0 ou 1). Cela est particulièrement utile lorsque plusieurs catégories peuvent coexister dans une même observation.

```
[4]: encoder = LabelBinarizer()
      encoder.fit_transform(y)
```

```
[4]: array([[1, 0, 0, 0],
           [0, 1, 0, 0],
           [1, 0, 0, 0],
           [0, 0, 0, 1],
           [0, 0, 1, 0],
           [0, 1, 0, 0],
           [1, 0, 0, 0]])
```

```
[5]: y = np.array(['chat', 'poils',
                   ['lion', 'poils'],
                   ['chien', 'poils'],
                   ['vache', 'cuire'],
                   ['chat', 'poils'],
                   ['oiseau', 'plumes']])

encoder = MultiLabelBinarizer()
encoder.fit_transform(y)
```

```
[5]: array([[1, 0, 0, 0, 0, 0, 1, 0],
           [0, 0, 0, 1, 0, 0, 1, 0],
           [0, 1, 0, 0, 0, 0, 1, 0],
           [0, 0, 1, 0, 0, 0, 0, 1],
           [1, 0, 0, 0, 0, 0, 1, 0],
           [0, 0, 0, 0, 1, 1, 0, 0]])
```

3.1.3 OrdinalEncoder et OneHotEncoder :

L'OrdinalEncoder est spécialement conçu pour l'encodage des variables catégorielles ordinales, c'est-à-dire celles qui présentent un ordre naturel entre les catégories. Il attribue des valeurs numériques à chaque catégorie tout en préservant leur ordre relatif. Cette méthode est préférable lorsque l'ordre des catégories a une signification pertinente, par exemple, les niveaux d'éducation (primaire, secondaire, tertiaire).

Le OneHotEncoder est largement utilisé pour traiter les variables catégorielles nominales. Il transforme chaque catégorie en une colonne binaire distincte, assignant une valeur de 1 à la catégorie présente et 0 aux autres. Cela permet d'éviter l'introduction d'un ordre artificiel entre les catégories et assure la compatibilité avec les algorithmes qui ne peuvent pas interpréter les valeurs numériques directement. Cette technique est particulièrement adaptée lorsque les catégories n'ont pas d'ordre intrinsèque.

```
[6]: X = np.array(['chat', 'poils'],
                 ['lion', 'poils'],
                 ['chien', 'poils'],
                 ['vache', 'cuire'],
                 ['chat', 'poils'],
                 ['oiseau', 'plumes']))

encoder = OrdinalEncoder()
encoder.fit_transform(X)
```

```
[6]: array([[0., 2.],
           [2., 2.],
           [1., 2.],
           [4., 0.],
           [0., 2.],
           [3., 1.]])
```

```
[7]: X = np.array(['chat', 'poils'],
                 ['lion', 'poils'],
                 ['chien', 'poils'],
                 ['vache', 'cuire'],
                 ['chat', 'poils'],
                 ['oiseau', 'plumes']))

encoder = OneHotEncoder(sparse=False)
encoder.fit_transform(X)
```

```
[7]: array([[1., 0., 0., 0., 0., 0., 0., 1.],
           [0., 0., 1., 0., 0., 0., 0., 1.],
           [0., 1., 0., 0., 0., 0., 0., 1.],
           [0., 0., 0., 0., 1., 1., 0., 0.],
           [1., 0., 0., 0., 0., 0., 0., 1.],
           [0., 0., 0., 1., 0., 0., 1., 0.]])
```

3.2 Normalisation

La standardisation, vise à mettre les valeurs de différentes variables à la même échelle, éliminant ainsi les biais potentiels introduits par des unités de mesure disparates. Ces techniques constituent des étapes essentielles pour assurer la qualité et la pertinence des données, facilitant ainsi leur traitement et leur interprétation ultérieurs dans le cadre de diverses analyses.

```
[8]: from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
```

```
[9]: from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
```

3.2.1 MinMaxScaler

MinMax Scaler : Le MinMax Scaler est une technique de mise à l'échelle qui transforme les données en les ramenant dans une plage spécifique, généralement entre 0 et 1. Il fonctionne en appliquant la formule :

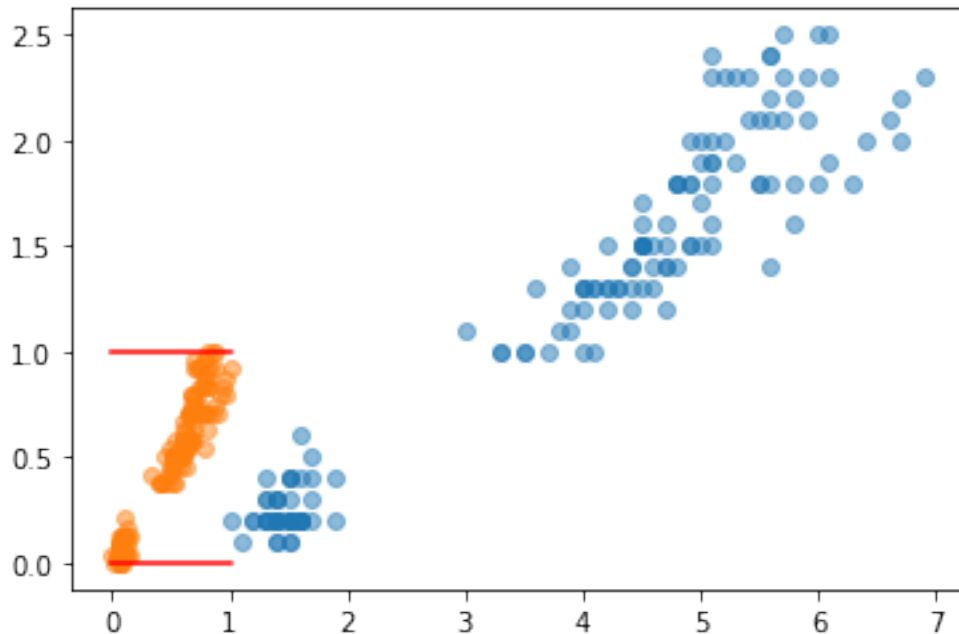
$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

où X_{min} est la valeur minimale de la variable, X_{max} est la valeur maximale de la variable et X est la valeur originale. L'avantage du MinMax Scaler est qu'il préserve la relation entre les données, tout en les ramenant à une plage standard. Cependant, il peut être sensible aux valeurs aberrantes, car il est directement influencé par les valeurs extrêmes.

```
[10]: X_minmax = MinMaxScaler().fit_transform(X)

plt.scatter(X[:, 2], X[:, 3], alpha=0.5)
plt.scatter(X_minmax[:, 2], X_minmax[:, 3], alpha=0.5)
plt.plot(np.array([0,1]), np.array([1,1]), c='r')
plt.plot(np.array([0,1]), np.array([0,0]), c='r')
```

```
[10]: [<matplotlib.lines.Line2D at 0x7fc47dca0820>]
```



3.2.2 StandardScaler

Le Standard Scaler, également connu sous le nom de Z-Score Scaler, transforme les données en utilisant la formule :

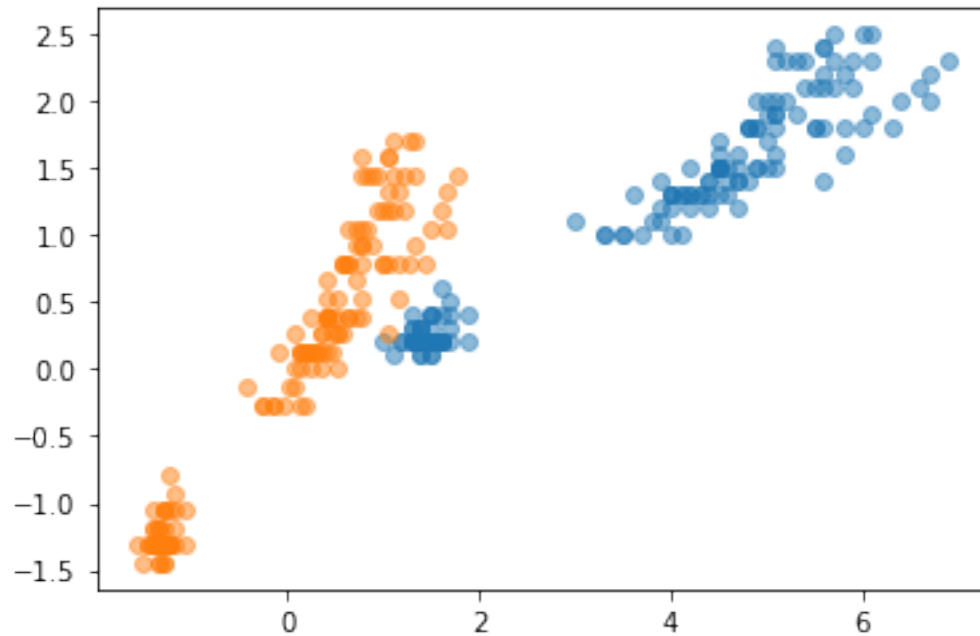
$$X_{scaled} = \frac{X - \mu}{\sigma}$$

où μ est la moyenne de la variable et σ est l'écart type. Cette technique centre les données autour de zéro et les met à l'échelle en fonction de l'écart type. Cela est bénéfique pour les algorithmes sensibles à l'échelle et à la distribution des données. Cependant, le Standard Scaler peut être influencé par les valeurs aberrantes, ce qui peut affecter la transformation.

```
[11]: X_stdsc1 = StandardScaler().fit_transform(X)

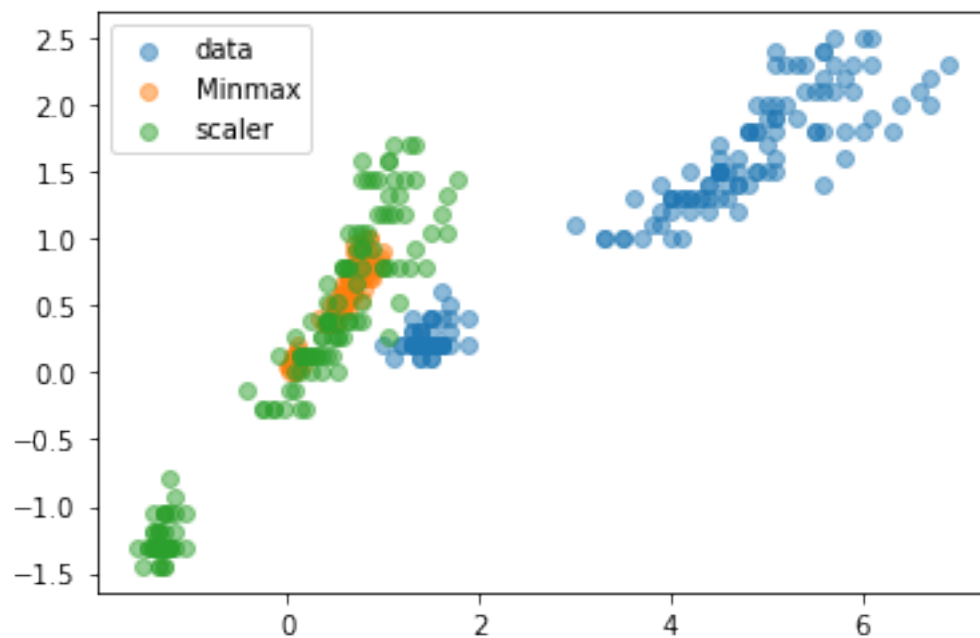
plt.scatter(X[:, 2], X[:, 3], alpha=0.5)
plt.scatter(X_stdsc1[:, 2], X_stdsc1[:, 3], alpha=0.5)
```

```
[11]: <matplotlib.collections.PathCollection at 0x7fc47ddb7880>
```



```
[12]: plt.scatter(X[:, 2], X[:, 3], alpha=0.5, label='data')
plt.scatter(X_minmax[:, 2], X_minmax[:, 3], alpha=0.5, label='Minmax')
plt.scatter(X_stdsc1[:, 2], X_stdsc1[:, 3], alpha=0.5, label='scaler')
plt.legend()
```

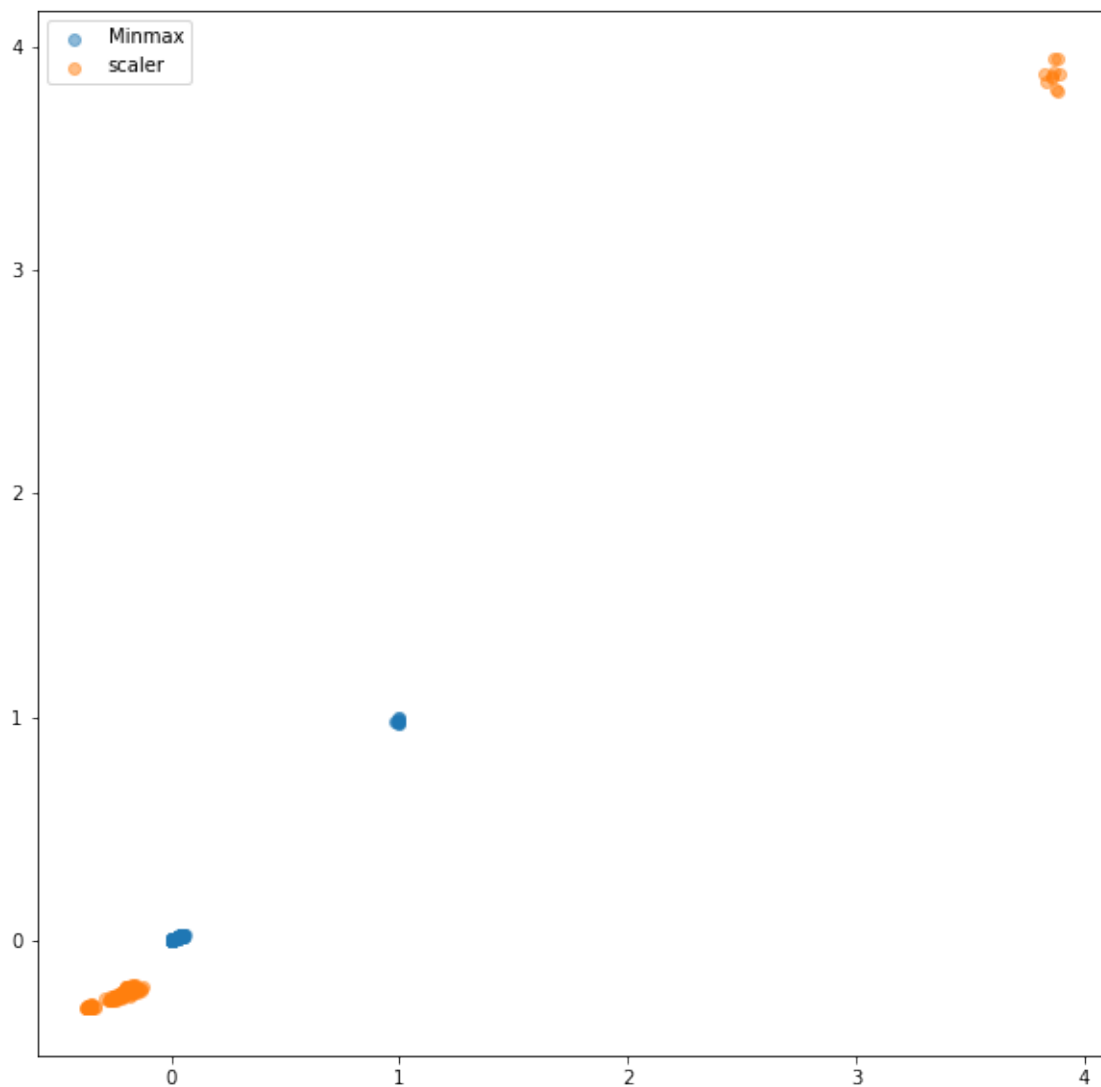
[12]: <matplotlib.legend.Legend at 0x7fc47de98dc0>




```
[13]: outliers = np.full((10,4),100)+ np.random.randn(10,4)
x = np.vstack((X,outliers))
X_minmax = MinMaxScaler().fit_transform(x)
X_stdsc1 = StandardScaler().fit_transform(x)
plt.figure(figsize=(10,10))

plt.scatter(X_minmax[:, 2], X_minmax[:, 3],alpha=0.5,label='Minmax')
plt.scatter(X_stdsc1[:, 2], X_stdsc1[:, 3],alpha=0.5,label='scaler')
plt.legend()
```

[13]: <matplotlib.legend.Legend at 0x7fc47df26fd0>



3.2.3 RobustScaler

Le Robust Scaler est une méthode de mise à l'échelle robuste aux valeurs aberrantes. Il utilise la médiane et l'intervalle interquartile (IQR) pour normaliser les données en utilisant la formule :

$$X_{scaled} = \frac{X - Q_1}{Q_3 - Q_1}$$

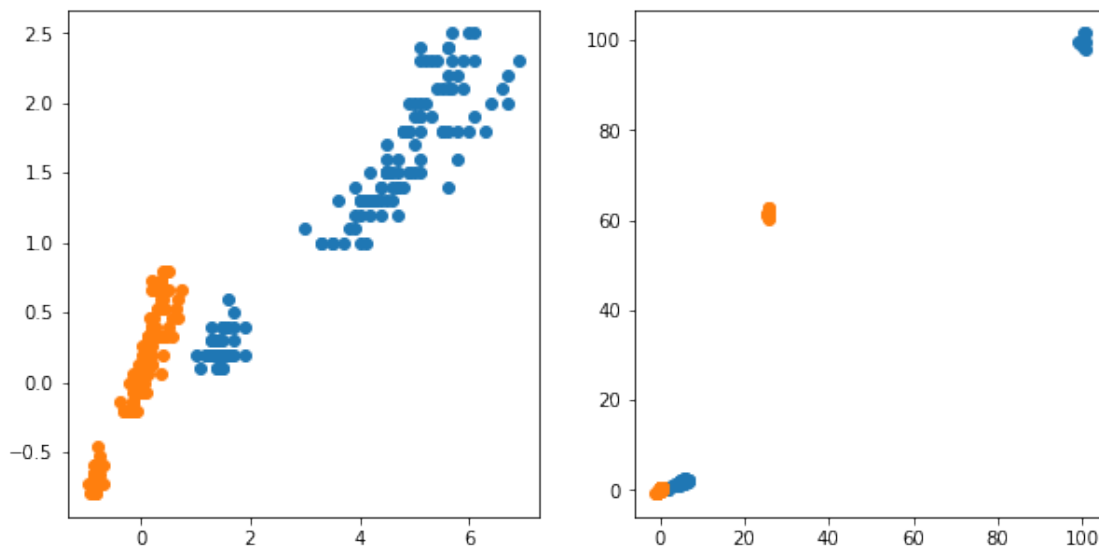
où Q_1 est le premier quartile et Q_3 est le troisième quartile. Le Robust Scaler est moins sensible aux valeurs aberrantes car il utilise des mesures de position centrale et de dispersion résistantes aux valeurs extrêmes. Cependant, il peut également réduire l'écart entre les données non aberrantes, ce qui peut avoir un impact sur la sensibilité des algorithmes.

Chacune de ces techniques de mise à l'échelle a ses avantages et ses inconvénients. Le choix de la méthode dépend de la distribution des données, de la présence de valeurs aberrantes et des exigences spécifiques de l'analyse.

```
[14]: X_robust = RobustScaler().fit_transform(X)
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.scatter(X[:, 2], X[:, 3])
plt.scatter(X_robust[:, 2], X_robust[:, 3])

X_robust = RobustScaler().fit_transform(x)
plt.subplot(1,2,2)
plt.scatter(x[:, 2], x[:, 3])
plt.scatter(X_robust[:, 2], X_robust[:, 3])
```

```
[14]: <matplotlib.collections.PathCollection at 0x7fc47e5dae50>
```



4 train_test_split

`train_test_split` est une étape cruciale lors de la préparation des données pour l'apprentissage automatique. Cette fonction est utilisée pour diviser un ensemble de données en deux parties distinctes : un ensemble d'entraînement (training set) et un ensemble de test (test set). Cette division permet de mesurer la performance d'un modèle d'apprentissage automatique de manière impartiale et réaliste.

Le but principal de cette séparation est d'évaluer la capacité du modèle à généraliser à de nouvelles données non vues auparavant. L'ensemble d'entraînement est utilisé pour ajuster les paramètres du modèle et apprendre les relations sous-jacentes dans les données. L'ensemble de test, quant à lui, sert à évaluer la performance du modèle sur des données qu'il n'a jamais rencontrées auparavant. Cela permet de détecter le surapprentissage (overfitting), où le modèle peut trop bien s'adapter aux données d'entraînement mais ne généralise pas correctement.

En divisant les données en ensembles d'entraînement et de test, on obtient une évaluation plus objective de la performance du modèle sur des données inconnues. Cela aide à ajuster les paramètres du modèle pour obtenir une meilleure généralisation. La fonction `train_test_split` prend en entrée les données à diviser, ainsi qu'un paramètre pour spécifier la proportion de données à allouer à l'ensemble de test. En divisant les données de cette manière, on améliore la fiabilité de l'évaluation du modèle et on augmente la confiance dans sa capacité à bien fonctionner sur de nouvelles données.

```
[15]: from sklearn.model_selection import train_test_split
```

```
[16]: import pandas as pd
df = pd.read_excel('Datasets/titanic3.xls')
df = df.drop(['name', 'sibsp', 'parch', 'ticket', 'fare', 'cabin', 'embarked',
            ↪ 'boat', 'body', 'home.dest'], axis=1)
df = df.dropna(axis=0)
df.head()
```

```
[16]:  pclass  survived    sex    age
0         1         1  female  29.0000
1         1         1   male   0.9167
2         1         0  female   2.0000
3         1         0   male  30.0000
4         1         0  female  25.0000
```

La fonction `train_test_split` est utilisée pour diviser un ensemble de données en ensembles d'entraînement et de test. Voici les arguments et paramètres importants : Arguments :

- `X` : Le tableau (ou dataframe) contenant les caractéristiques (variables indépendantes) de l'ensemble de données.
- `y` : Le tableau contenant les étiquettes (variable dépendante) correspondant aux caractéristiques de l'ensemble de données.

Paramètres :

- `test_size` : Le paramètre `test_size` détermine la proportion des données à allouer à l'ensemble de test. Par exemple, `test_size=0.2` allouera 20% des données à l'ensemble de test et 80%

à l'ensemble d'entraînement. `train_size` : Ce paramètre permet de spécifier directement la taille de l'ensemble d'entraînement. Il est généralement utilisé en alternative à

- `test_size` lorsque l'on souhaite définir explicitement la taille de l'ensemble d'entraînement.
- `random_state` : Le paramètre `random_state` permet de fixer une graine pour la génération aléatoire des séparations des ensembles. Cela garantit que les séparations restent constantes lors de différentes exécutions du code.
- `shuffle` : Ce paramètre indique si les données doivent être mélangées avant la division. Par défaut, il est généralement défini sur `True` pour assurer que les données sont mélangées aléatoirement.
- `stratify` : Lorsque les données sont déséquilibrées, ce paramètre permet de garantir que les proportions des classes dans l'ensemble d'entraînement et l'ensemble de test restent similaires à celles de l'ensemble de données original.

```
[17]: y = df['survived']
X = df.drop(['survived', 'sex'], axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

Ce code représente la mise en œuvre d'un classificateur K plus proches voisins (KNN). Le modèle KNN est créé et entraîné avec les données d'entraînement `X_train` et les étiquettes correspondantes `y_train`. Ensuite, son efficacité est évaluée en utilisant les données de test `X_test` et `y_test`, et le score de précision est calculé pour mesurer sa performance. Ce score est une mesure de la précision du modèle dans la classification des données de test.

```
[18]: from sklearn.neighbors import KNeighborsClassifier
KNN = KNeighborsClassifier()
KNN.fit(X_train, y_train)
KNN.score(X_test, y_test)
```

```
[18]: 0.6564885496183206
```

5 Exercice

Mettez en pratique ce que vous avez appris sur les techniques de prétraitement en utilisant le jeu de données Titanic. - Commencez par utiliser un “imputer” pour gérer les valeurs manquantes. - Ensuite, appliquez une “mise à l'échelle” pour normaliser vos données. - Enfin, explorez les “encodeurs” pour gérer les variables catégorielles.

Ensuite, entraînez l'algorithme des k plus proches voisins (KNN) avec différentes configurations. Votre objectif est de déterminer quelles techniques de prétraitement donnent les meilleurs résultats en termes de précision de prédiction.

6 Transformation de Colonnes et Création de Pipelines

Scikit-Learn, offre des outils puissants pour effectuer des transformations. Deux de ces outils sont la transformation de colonnes et les pipelines.

6.1 Transformation de Colonnes

La transformation de colonnes consiste à appliquer des opérations spécifiques à une ou plusieurs colonnes de votre ensemble de données. Cela peut inclure des opérations telles que le remplacement des valeurs manquantes, la normalisation des valeurs numériques ou l’encodage des variables catégorielles.

Par exemple, si vous avez une colonne “Âge” avec des valeurs manquantes, vous pouvez utiliser un “imputer” pour remplir ces valeurs manquantes par la moyenne de l’âge des autres individus. Pour les colonnes catégorielles comme “Genre”, vous pouvez utiliser un encodeur pour les convertir en valeurs numériques.

6.2 Pipelines

Un pipeline est un ensemble ordonné d’étapes de transformation que vous pouvez appliquer séquentiellement à vos données. Cela permet de simplifier considérablement le processus de prétraitement et d’entraînement de modèles.

Par exemple, vous pourriez créer un pipeline qui effectue les étapes suivantes dans cet ordre : 1. Remplacement des valeurs manquantes dans les colonnes numériques. 2. Encodage des variables catégorielles. 3. Mise à l’échelle des données numériques.

Une fois que vous avez défini ce pipeline, vous pouvez l’appliquer à vos données en une seule ligne de code, ce qui le rend très efficace et réutilisable.

```
[19]: df = pd.read_excel('Datasets/titanic3.xls')
df = df.drop(['name', 'sibsp', 'parch', 'ticket', 'cabin', 'embarked', 'boat', 'body', 'home.dest'], axis=1)
df = df.dropna(axis=0)

y = df['survived']
X = df.drop('survived', axis=1)
numerical_features = ['pclass', 'age', 'fare']
categorical_features = ['sex']

X_train, X_test, y_train, y_test = train_test_split(X, y)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
[19]: ((783, 4), (262, 4), (783,), (262,))
```

```
[20]: from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier
```

Ce code est une illustration de la création de pipelines et de la transformation de colonnes.

1. **numerical_pipeline** (pipeline numérique) : Ce pipeline est destiné à traiter les données numériques. Il comprend deux étapes :

- `SimpleImputer()` : remplit les valeurs manquantes dans les données numériques en utilisant la moyenne par défaut.
 - `StandardScaler()` : standardise les données numériques .
2. `categorical_pipeline` (pipeline catégorique) : Ce pipeline est conçu pour traiter les données catégorielles. Il comprend deux étapes :
 - `SimpleImputer(strategy='most_frequent')` : remplit les valeurs manquantes en utilisant la stratégie “most_frequent”.
 - `OneHotEncoder()` : Cette étape encode les données en utilisant la technique “one-hot encoding”.
 3. `preprocessor` (pré-processeur) : est créé en utilisant `make_column_transformer`. Il prend deux paramètres :
 - Le premier paramètre est le `numerical_pipeline` et la liste `numerical_features` qui contient les noms des colonnes numériques.
 - Le deuxième paramètre est le `categorical_pipeline` et la liste `categorical_features` qui contient les noms des colonnes catégorielles.

En utilisant ce `preprocessor`, vous pouvez appliquer ces transformations aux données d’entraînement et de test de manière cohérente, ce qui facilite grandement la préparation des données pour l’entraînement de modèles d’apprentissage automatique.

```
[21]: numerical_pipeline = make_pipeline(SimpleImputer(), StandardScaler())
categorical_pipeline = make_pipeline(SimpleImputer( strategy='most_frequent'),
↪ OneHotEncoder())
preprocessor = make_column_transformer((numerical_pipeline, numerical_features),
↪ (categorical_pipeline, categorical_features))
```

Ces extraits de code montrent comment créer un modèle d’apprentissage automatique en utilisant un pipeline pour le prétraitement des données et un algorithme `KNeighborsClassifier`.

`model = make_pipeline(preprocessor, KNeighborsClassifier())` : Un modèle est créé en utilisant `make_pipeline`. Ce modèle comprend deux étapes : - La première étape est le préprocesseur `preprocessor` que nous avons défini précédemment. - La deuxième étape est l’algorithme `KNeighborsClassifier`.

```
[22]: model = make_pipeline(preprocessor, KNeighborsClassifier())
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

```
[22]: 0.8053435114503816
```

7 Exercice

Pour cet exercice, nous allons utiliser un ensemble de données sur les voitures celui utilisé lors du précédent TP. L’objectif est de construire un modèle prédictif pour estimer `enginetype` Type de moteur d’un véhicule en fonction de ses caractéristiques.

Voici les étapes à suivre :

1. **Préparation des données** : Commencez par charger l’ensemble de données sur les voitures.

Vous devrez sélectionner la variable cible, qui sera la puissance (horsepower), ainsi que les caractéristiques pertinentes à utiliser pour la prédiction.

2. **Prétraitement des données** : Créez un pipeline de prétraitement des données qui comprendra les étapes suivantes :
 - Imputation des éventuelles valeurs manquantes.
 - Normalisation des caractéristiques numériques pour mettre toutes les variables à la même échelle.
 - Encodage des variables catégorielles si nécessaire.
3. **Modélisation avec k-NN** : Utilisez l'algorithme des k plus proches voisins (k-NN) pour construire un modèle de prédiction de la puissance des voitures. Expérimentez avec différentes valeurs de k pour trouver la meilleure performance.
4. **Évaluation du modèle** : Évaluez la performance du modèle en utilisant des métriques appropriées,

L'objectif de cet exercice est de créer un modèle de prédiction capable d'estimer avec précision la puissance des voitures en fonction de leurs caractéristiques.